Practical 11

Represent a real-world problem using propositional logic or first-order logic and perform logical inference to derive conclusions using an AI reasoning approach.

Objective

To represent the classical 'West is a criminal' reasoning problem using propositional Horn-clause logic and to derive the conclusion criminal(West) from the knowledge base using a forward-chaining inference engine.

Procedure

1. Encode the domain facts as ground literals: american(West), missile(M1), owns(Nono, M1), sells(West, M1, Nono), enemy(Nono, America).
2. Encode the domain rules as Horn clauses: missile(x) -> weapon(x); enemy(x, America) -> hostile(x); american(x) & weapon(y) & sells(x,y,z) & hostile(z) -> criminal(x).
3. Implement each rule as a Python function that scans the current fact set and returns any new facts it can derive.
4. Implement forward_chain(): repeatedly apply every rule to the growing fact set, adding any newly derived facts, until no rule produces anything new or the goal fact is derived.
5. Set the goal to criminal(West) and run forward chaining, printing every rule firing and the new fact(s) it derives at each iteration.
6. Report whether the goal was proven, and print the final, fully-expanded knowledge base.

Program / Code

```
# ================================================================
# PRACTICAL 11
# Propositional Logic Inference - Forward Chaining
# Domain: "Who is a criminal?" (classic AI textbook example)
# ================================================================

# ----------------------------------------------------------------
# Knowledge Base expressed as Horn clauses:
#   IF american(x) AND weapon(y) AND sells(x, y, z) AND hostile(z)
#      THEN criminal(x)
#   IF missile(x) THEN weapon(x)
#   IF enemy(x, America) THEN hostile(x)
#
# Facts:
#   american(West)
#   missile(M1)
#   sells(West, M1, Nono)
#   enemy(Nono, America)
# ----------------------------------------------------------------

facts = {
```

```
    ("american", "West"),
    ("missile", "M1"),
    ("owns", "Nono", "M1"),
    ("sells", "West", "M1", "Nono"),
    ("enemy", "Nono", "America"),
}

# Rules are represented as (premises, conclusion_template, variable_bindings_fn)
# For simplicity in this toy engine, we hard-code the 3 domain rules and
# apply forward chaining generically over variable substitutions.

def rule_weapon(facts):
    """missile(x) -> weapon(x)"""
    new_facts = set()
    for f in facts:
        if f[0] == "missile":
            new_facts.add(("weapon", f[1]))
    return new_facts

def rule_hostile(facts):
    """enemy(x, America) -> hostile(x)"""
    new_facts = set()
    for f in facts:
        if f[0] == "enemy" and f[2] == "America":
            new_facts.add(("hostile", f[1]))
    return new_facts

def rule_criminal(facts):
    """american(x) & weapon(y) & sells(x, y, z) & hostile(z) -> criminal(x)"""
    new_facts = set()
    americans = {f[1] for f in facts if f[0] == "american"}
    weapons = {f[1] for f in facts if f[0] == "weapon"}
    hostiles = {f[1] for f in facts if f[0] == "hostile"}
    for f in facts:
        if f[0] == "sells":
            x, y, z = f[1], f[2], f[3]
            if x in americans and y in weapons and z in hostiles:
                new_facts.add(("criminal", x))
    return new_facts


RULES = [
    ("missile(x) -> weapon(x)", rule_weapon),
    ("enemy(x, America) -> hostile(x)", rule_hostile),
    ("american(x) & weapon(y) & sells(x,y,z) & hostile(z) -> criminal(x)", rule_criminal),
]


def forward_chain(facts, rules, goal):
    facts = set(facts)
    iteration = 0

    print("Initial Facts (KB):")
    for f in sorted(facts):
        print("  ", f)

    while True:
```

```
        iteration += 1
        new_facts = set()
        fired_this_round = []

        for rule_str, rule_fn in rules:
            produced = rule_fn(facts) - facts
            if produced:
                fired_this_round.append((rule_str, produced))
                new_facts |= produced

        if not new_facts:
            break

        print(f"\nIteration {iteration}:")
        for rule_str, produced in fired_this_round:
            print(f"  Rule fired : {rule_str}")
            for p in sorted(produced):
                print(f"     derived  -> {p}")

        facts |= new_facts

        if goal in facts:
            break

    return facts


if __name__ == "__main__":
    print("=" * 65)
    print("   PROPOSITIONAL LOGIC INFERENCE - FORWARD CHAINING")
    print("   Domain: 'West is a criminal' (classic KB)")
    print("=" * 65)

    goal = ("criminal", "West")

    final_facts = forward_chain(facts, RULES, goal)

    print("\n" + "-" * 65)
    if goal in final_facts:
        print(f"GOAL PROVEN: {goal} is entailed by the knowledge base.")
    else:
        print(f"GOAL NOT PROVEN: {goal} could not be derived.")

    print("\nFinal Knowledge Base (all facts + derived facts):")
    for f in sorted(final_facts):
        print("  ", f)

    print("=" * 65)
```

Output

```
=================================================================
   PROPOSITIONAL LOGIC INFERENCE - FORWARD CHAINING
   Domain: 'West is a criminal' (classic KB)
=================================================================
```

```
Initial Facts (KB):
   ('american', 'West')
   ('enemy', 'Nono', 'America')
   ('missile', 'M1')
   ('owns', 'Nono', 'M1')
   ('sells', 'West', 'M1', 'Nono')

Iteration 1:
  Rule fired : missile(x) -> weapon(x)
     derived  -> ('weapon', 'M1')
  Rule fired : enemy(x, America) -> hostile(x)
     derived  -> ('hostile', 'Nono')

Iteration 2:
  Rule fired : american(x) & weapon(y) & sells(x,y,z) & hostile(z) -> criminal(x)
     derived  -> ('criminal', 'West')

----------------------------------------------------------------
GOAL PROVEN: ('criminal', 'West') is entailed by the knowledge base.

Final Knowledge Base (all facts + derived facts):
   ('american', 'West')
   ('criminal', 'West')
   ('enemy', 'Nono', 'America')
   ('hostile', 'Nono')
   ('missile', 'M1')
   ('owns', 'Nono', 'M1')
   ('sells', 'West', 'M1', 'Nono')
   ('weapon', 'M1')
================================================================
```

Conclusion

Forward-chaining inference successfully derived criminal(West) from the initial knowledge base in two iterations: first deriving weapon(M1) from missile(M1), and hostile(Nono) from enemy(Nono, America); then, with weapon(M1) and hostile(Nono) now known, firing the criminal rule to derive criminal(West) since West is American, M1 is a weapon, West sells M1 to Nono, and Nono is hostile. This is a direct, worked demonstration of monotonic logical entailment: each rule firing only ever adds new true facts and never retracts existing ones, and the algorithm terminates once a fixed point (no new derivable facts) is reached or the goal is found — the same underlying mechanism used by rule-based expert systems and Prolog-style logic programming for automated reasoning over first-order/propositional knowledge bases.